

Project for the course of Introduction to Robotics

Redundancy Resolution for Circular End-Effector Motion

In this project, you will control a 7-degree-of-freedom Franka Emika Panda manipulator so that its end effector follows a circular trajectory in Cartesian space. The robot has more joints than strictly necessary to accomplish this task, and this redundancy will be used to improve the quality of the motion.



The project is designed to guide you from the mathematical formulation of inverse differential kinematics to its implementation in a realistic robotic software framework. You will first develop and validate your solution in MATLAB, and then port the same controller to a ROS2-based system.

The trajectory

The motion to be achieved is a **circle in Cartesian space of radius $r=0.4$ m**. You are free to choose the plane and position of the circle, a typical choice is a horizontal circle at a fixed height. The trajectory shall be executed in 1 second

The orientation of the end effector should be kept constant throughout the motion, so that the fingers of the gripper are kept normal to the plan of the circle and the **position of the end effector is variable in time**.

The objective is not only to follow this trajectory accurately, but also to understand how different joint configurations can produce the same end-effector motion.

Control idea

You will implement a controller based on **inverse differential kinematics**, where joint velocities are computed from the desired Cartesian motion using the robot Jacobian.

This term alone is sufficient to track the circle. However, since the robot is redundant, there are infinitely many joint velocity solutions that achieve the same end-effector motion. This is where the project becomes interesting.

You will extend the controller by adding a **null-space term**, which does not affect the end-effector motion but modifies the internal posture of the robot:

The second term allows you to shape how the robot moves while still perfectly tracking the circle.

Secondary objectives

Using the null-space term, you will introduce one or more additional objectives.

A natural choice is to keep the joints away from their limits. Even if the end-effector motion is correct, some solutions may push the robot toward uncomfortable or unsafe configurations. By introducing a suitable function that is maximized when joints are centered in their range, the robot can maintain a more balanced posture.

Another meaningful objective is to maximize the **manipulability**, which measures how easily the robot can move in different directions. Near singular configurations, the robot loses the ability to move freely; maximizing manipulability helps avoid such situations.

The key idea is that all these behaviors emerge **without changing the end-effector trajectory**, but only by exploiting redundancy.

MATLAB implementation (max 3 points)

The first part of the project focuses on developing and testing the controller in MATLAB, using the robotics toolbox. Here you can work in a controlled environment, where you compute the Jacobian, define the trajectory, and simulate the robot motion.

Joint velocities are computed using the controller and then integrated numerically to obtain joint positions. Through simulation, you will observe how the robot behaves when only the tracking term is used, and how the behavior changes when the null-space term is introduced.

You should visualize the trajectory of the end effector, the tracking error, and the evolution of the joint configuration. In particular, it is interesting to observe how different choices of the secondary objective affect the internal motion of the robot while leaving the Cartesian trajectory unchanged.

ROS2 implementation (max 2 additional points)

In the second part of the project, the same control logic is implemented within a ROS2 framework. The controller becomes a node that reads the current joint configuration and outputs commands to the robot.

You can control the robot using joint velocities directly, and by integrating velocities to generate joint position references. You can compare the two approaches if both position and velocity interfaces are available.

This step introduces practical aspects such as communication, timing, and system integration. The goal is to reproduce in ROS2 the same behavior observed in MATLAB, highlighting the transition from simulation to a more realistic robotic setup.

What you should observe

At the end of the project, you should clearly see that:

- the end effector follows the same circular trajectory in all cases,
- different redundancy strategies produce different joint motions,
- the robot can avoid joint limits or singular configurations without affecting the main task.

This illustrates one of the key ideas in robotics: **the same task can be achieved in many different ways, and good control design allows us to choose the best one.**

Deliverables

You are expected to submit your MATLAB implementation, your ROS2 package, and a short report describing your approach and results. The report should explain the control strategy, the chosen secondary objective, and the differences observed between MATLAB and ROS2.